

— EV CHARGER DATA PLATFORM

Turning chaotic charger data into accurate, on-time PSC reporting.

An AI-powered pipeline that cleans, validates, reconciles, and reports on
EV-charger data from third-party integrators — built end-to-end and
proven on Con Edison's own territory data.

258,343

RAW RECORDS
PROCESSED

254,414

GOLDEN RECORDS
OUT

4,105

DUPLICATES
RESOLVED

2.9 min

END-TO-END
RUNTIME

SECTION 1

The Problem

Why EV-charger data is a reporting liability today — and why it doesn't have to be.

Con Edison's EV-charger programs depend on external contractors whose data arrives through third-party integrators. It is frequently **late, incomplete, inconsistent, and unstructured** — and today, turning it into a regulatory report is a **manual** process that strains PSC mandates to report accurately and on time.



Charging Sessions

CSV logs — date-format chaos, W/kW unit swaps, missing fields.



Station Registry

GeoJSON dumps with free-text connector names and conflicting coordinates.



Contractor Reports

Unstructured email and fault digests — only a human or LLM can read them.

SECTION 2

Background Research

We studied how charger data actually behaves across the industry.

Across public datasets and industry material, one pattern dominates: **the mess is in the data you already have**, not in exotic formats. Unit swaps, date chaos, free-text enums, and late-arriving files are the norm. The fix had to be a **pipeline of small, testable stages** — not one clever model.

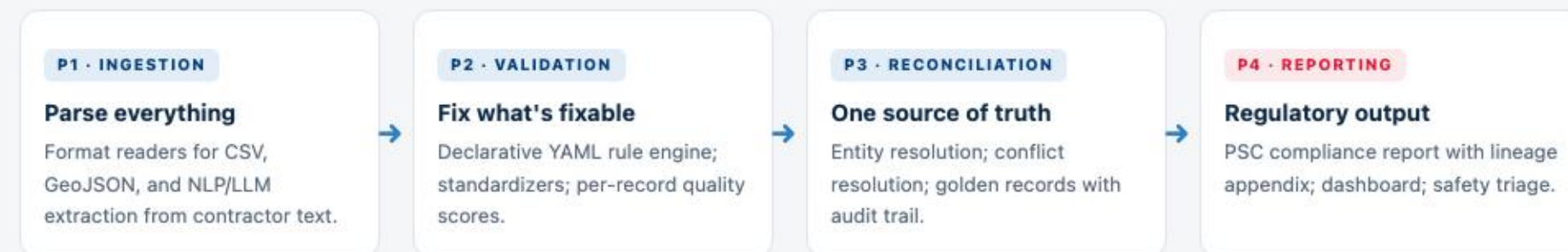


One surprising fact. In the official NYC DOT dataset — supposedly clean municipal data — our pipeline still found **4,105 duplicate records, 2,731 conflicting field values, and 3 unresolved SAFETY maintenance events** hiding in plain sight. The cost of not looking is regulatory.

SECTION 3

Ideation Process

Five phases, designed to run in parallel without ever blocking each other — then merged into one pipeline.



The enabling decisions

A frozen canonical schema and CLI contract let five builders work in parallel on separate branches. A fifth phase (P5) wraps it all in an orchestrator with health monitoring and an integration harness.

SECTION 4

What Failed

The honest list. Every item below was found, fixed, and re-tested.

✕ Contract-protocol deviations

Two stages emitted metrics only under a flag. Integration harness caught both.

✕ Config bug that looked correct

"urgent-safety" → MAJOR instead of SAFETY. Caught by hand-verifying fixtures.

✕ Reconciliation was too aggressive

First pass merged different drivers and chargers. Audit caught it; fix dropped runtime from 74 min → 2.2 min.

✕ Near-public privacy leak

Git auto-configured commit author with NYU username. Caught before push.

SECTION 5

What Worked — The Solution

One command, four stages, a full audit trail. Left: raw input. Right: golden output.

⚠ BEFORE — RAW INPUT

```
Source A · CSV      "08/21/2025","19:10:54","720.3","NULL"
Source B · AFDC     "NISSAN FAST CHR RIVERHEAD"
                  ev_connector_types: ["CHADEMO","J1772COMBO"]
                  open_date: 1327968000000
Source C · Contractor complaint via 311
                  location: "JON - Jerome 190th St"
                  charge box EVB-P2042308: screen dead
                  priority: urgent-safety
```

✓ AFTER — GOLDEN RECORD

```
Source A · Session  "2025-08-21T19:10:54-04:00"
                  "0.7203", "41.369", "PAID"
Source B · Charger   "NISSAN FAST CHR RIVERHEAD"
                  connectors: "J1772, CHAdemo", date: "2012-01-31"
Source C · Maintenance COMPLAINT · SAFETY
                  box EVB-P2042308
                  ← NLP/LLM-extracted
```

merged_from audit trail · 4,105 duplicates resolved · 2.9 min total runtime · 2,731 conflicts resolved

— STAGE 1 OF 4

Ingestion — Three formats in, one canonical format out

Tracing a real 311 complaint from Jerome Avenue. The ingestion stage uses **NLP** and **LLM extraction** to pull structured events from contractor free text, alongside CSV and GeoJSON readers. Two paths: a deterministic regex engine (default, offline, zero API cost) and a Claude LLM path with automatic regex fallback.

258,343 → 258,519

RECORDS IN → RECORDS OUT (+176 MUNICIPAL CHARGE BOXES CREATED)

⚠ RAW INPUT

```
// Source C — Contractor free-text report
customer complaint via 311, logged 2025-06-24
location given: "JON — Jerome 190th Street
—Municipal Parking"
charge box EVB-P2042308: screen dead
routed to: J. Alvarez
priority: urgent-safety
```



✓ PARSED RECORD

```
// canonical.jsonl
{ "record_type": "maintenance",
  "event_type": "COMPLAINT",
  "severity": "SAFETY",
  "charge_box": "EVB-P2042308",
  "location": "JON - Jerome 190th St",
  "date": "2025-06-24",
  "provenance": { "source": "contractor" } }
```

Validation — Fix what's fixable, flag what isn't

A declarative rule engine runs every record through standardizers. Dates → ISO-8601. Units repaired. Enums normalized. Missing values flagged in-place. **0 records quarantined. 251,773 issues flagged.**

0

RECORDS QUARANTINED — ALL ISSUES FLAGGED IN-PLACE

"00/21/2025 19:10:54.0000000" → 2025-08-21T19:10:54-04:00	ISO-8601 + NY offset
"720.3" (watts) → 0.7203 kW	unit repaired
"NULL" → ⚠ flagged · not dropped	missing field logged
urgent-safety → SAFETY	enum normalized
1327960000000 → 2012-01-31	Unix ms → date

Reconciliation — Duplicates merged into one source of truth

4 AFDC registry entries for the same charger → 1 golden record. Session dedup requires **same driver + same charger + same time window**. Every merge reversible via merged_from. **Audited: 96.5% legitimate.**

258,519 → 254,414

PRE-RECONCILIATION → GOLDEN RECORDS (4,105 MERGED)



Reporting — Regulator-ready output, one command

PSC compliance report with full data-lineage appendix. Every record traceable. 2.9 min end-to-end.



PSC COMPLIANCE REPORT — Con Edison EV Charger Data

4,219

CHARGERS DEPLOYED

250,181

CHARGING SESSIONS

5.69 GWh

ENERGY DELIVERED

82.1%

MUNICIPAL UPTIME

89.4%

DATA COMPLETENESS

3

SAFETY EVENTS

Lineage appendix traces every record back to its source · `python -m src.platform.run --pipeline full --no-llm`

— SECTION 6

Testing — Baseline, Then the Hard Case

First a controlled seed dataset. Then the official NYC DOT dataset — real, uncurated, in Con Edison territory.

PHASE 1 · SEED DATASET

Cary, NC · 20k+ records

Synthetic messiness planted to stress every rule.

76

FIXTURE TESTS

100%

PROBLEMS CAUGHT

✓ Every planted defect detected. **Never silently dropped a record.**

PHASE 2 · NYC DOT OFFICIAL DATA

Con Edison territory · 258k records

Real municipal garage data + AFDC registry + contractor reports.

26.9s

VALIDATION

2.2 min

RECONCILIATION

4,105

DUPLICATES RESOLVED

3

SAFETY EVENTS

✓ Flat-memory streaming, deterministic re-runs. **298 tests pass offline.**

— SECTION 7

Our Recommendation

Five steps to turn this into Con Edison's reporting backbone.

- 1 Adopt the pipeline as the PSC reporting path.** Run on every batch; lineage appendix makes "accurate and on time" auditable.
- 2 Start with validation + reporting, LLM off by default.** The deterministic path delivers most value at zero marginal cost.
- 3 Keep a human in the loop for SAFETY anomalies only.** Everything else auto-resolves with logged decisions.
- 4 Instrument it.** Per-stage health metrics ship with every run.
- 5 Then extend.** Contractor-portal feeds, nightly scheduling. Reconciliation at 2.2 min — scale is no longer the barrier.

— SECTION 8

What Could Go Wrong

We named the risks so they can be managed instead of discovered.

Model & provider drift

LLM quality changes. Mitigation: pinned model config, deterministic fallback.

Schema drift

New integrator format shifts interpretation. Proposal process must be run.

Late-arriving data

A file after the report window forces a restatement. Pipeline supports it.

Scale

2.2 minutes at 258k records. Full-state rollout keeps same architecture.

Overtrust in scores

A 0.99 quality score is not ground truth. Conflict logs show why.

Credential hygiene

Keys live only in environment; pipeline scans every submission.

— SECTION 9

Questions for Con Edison

We built this for you — now help us understand how it fits into your reality.

- 1 **Do you already use AI or NLP in your data workflows today?** If so, where — and what's working versus what isn't?
- 2 **How much of the current data process is human versus automated?** Spreadsheet reconciliation? Manual report assembly?
- 3 **How many datasets do you typically merge into one report?** We demonstrated three — your reality likely involves more.
- 4 **What would it take to roll out a system like this?** Who approves it? What does the pilot process look like?